Working with Bitwise AND (&), Bitwise OR (|) and Bitwise X-OR(^)

---

a) Logical Operator works with boolean value only.
Example: IO.println(5 && 6); //error

b) Bitwise AND (&), Bitwise OR (|) and Bitwise X-OR (^) can work with number and boolean both types of values.

Example:

```
IO.println(5 & 6); //Bitwise AND (&)     //Valid
IO.println(true & true); //Bitwise AND(&)  //Valid
```

//Program on Bitwise OR Operator

Bitwise OR, where just like **Logical OR** at-least one condition must be true as well as It is known as non-short circuit operator because, If the first condition is true, still It will verify all the right-side condition.

BitwiseOr.java

```
//Bitwise OR [non short circuit behaviour]

void main()
{
 int z = 5;
 if(++z > 5 || ++z > 6)   //6 > 5 //Logical OR
 {
  z++;
 }
 IO.println(z);  //7

 IO.println("...............");

 z = 5;
 if(++z > 5 | ++z > 6)   //6 > 5  Bitwise OR
 {
   z++;
 }
 IO.println(z); //8

}
```

BitwiseAnd.java

```
//Bitwise AND [non short circuit behaviour]

void main()
{
    int z = 5;
  if(++z > 6 & ++z> 6)
  {
    IO.println("Inside If");
    z++;
  }
  IO.println(z); //7

}
```

Bitwise X-OR (^)

Truth table of AND (A.B), OR (A+B) and X-OR (A⊕B) Gate :

A	B	AND(A.B)	OR (A+B)	X-OR (A⊕B)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

⇩ Same Input ouput is 0 i.e false

BitwiseX_OR.java

```
//Bitwise X-OR [Behaviour with boolean ]

void main()
{
     IO.println(true ^ true); //false        Same inputs output is false
     IO.println(true ^ false); //true        Alternate inputs output is true
     IO.println(false ^ false); //false
     IO.println(false ^ true); //true
}
```

Bitwise Operator with numeric values :

```
//Bitwise X-OR [Behaviour with numeric ]

void main()
{
     IO.println(5 & 7); //5
     IO.println(5 | 7); //7
     IO.println(5 ^ 7); //2
}
```

Binary of 5	:	1	0	1
Binary of 7	:	1	1	1
[5 & 7]	:	1	0	1
[5 | 7]	:	1	1	1
[5 ^ 7]	:	0	1	0

The result will come based on the **truth table** but not binary calculation

Bitwise Complement Operator (~)

* It will work with only numeric value **but not boolean.**
  Example :
  IO.println(~ true);   //Invalid
* It is used to invert the number that means positive number will become negative & negative will positive
* The result can be calculated using the formula :

~ x = - (x+1)

Complement.java

```
//Bitwise Complements Operator(~)  [~x = -(x+1)]

void main()
{
     IO.println(~ 5);   // -6
     IO.println(~ -4);  // 3
}
```

Bitwise shift operator :

It will work with two operands
It is used to work at the binary (bit) level, not on decimal values.
It is used to shift the bits of an integer value to the left (<<) or right (>>).
They are mainly used for fast arithmetic operations at bit level.

x<<n = x * 2^n
x>>n = x / 2^n

Left shift Operator formula: x<<n = x * 2ⁿ
Right shift Operator formula: x>>n = x / 2ⁿ

Shift.java

```
//Bitwise Shift Operator

void main()
{
     IO.println(9<<3);//72

     IO.println(9>>2);// 2
}
```

Binary Numeric Promotion in java:

While working with **Arithmetic Operators**, **Unary minus, plus** operator and **Bitwise operators** result is promoted to int type so to hold the expression result minimum int data type (32 bits required).

Numeric Promotion Chart :

Operand1	Operand2	Promoted Type (Result)
byte	byte	int
byte	short	int
short	short	int
char	char	int
int	long	long
long	float	float
float	double	double

Promotion.java

```
//Binary Numberic Promotion in Java


void main()
{
     short x = 12;
     x += 1; //Valid because compound operator does not have numeric promption
     IO.println(x);


   char ch = 'A'  + 'B';  //Compile time constant


   char a = 'A';
     char b = 'B';
     //char c = a + b;  //Invalid


   final char p = 'A';
     final char q = 'B';
     char r = p + q;

}
```

Automatic binary numberic promotion is required to prevent from data loss.

Promotion2.java

```
//Binary Numberic Promotion in Java


void main()
{
     byte b = 12;
     byte c = 15;
     int res = b + c;

     IO.println("........");
     int y = -b;

     IO.println("........");
     int z = b & c;

}
```

Limitation of if else:

The major drawback with if condition is, it checks the condition **again and again** so it increases the burden over CPU hence we introduced switch-case statement to reduce the overhead of the CPU.

Switch Statement in java (Till JDK 17V):

It is a **selective (decision making) statement** in java so, we can select one case among the available cases.

While writing the cases, break is **optional** but if we use break then the control will move **from out of the switch body.**

We can write default so if any statement is not **matching** then default will be executed but it is optional.

In the new switch case **break and :** both are **not required**, We can replace with **->** **s**ymbol

If we have multiple statements then {} are compulsory.

Old Switch Case :

```
void main()
{
     int choice = Integer.parseInt(IO.readln("Enter a number :"));

       switch(choice)
     {
          case 1 :
               IO.println("Case1");
             break;
          case 2 :
               IO.println("Case2");
             break;
          case 3 :
               IO.println("Case3");
              break;
          case 4 :
               IO.println("Case4");
              break;
          default :
               IO.println("Invalid");

     }


}
```

New switch case :

```
//switch case

void main()
{
     int choice = Integer.parseInt(IO.readln("Enter a number :"));

   switch(choice)
     {
          case 1 ->
          {
          IO.println("Case 1");
          IO.println("Sub case");
          }
          case 2 -> IO.println("Case 2");
          case 3 -> IO.println("Case 3");
          case 4 -> IO.println("Case 4");
          case 5 -> IO.println("Case 5");
          //default -> IO.println("Invalid");

     }


}
```